

DATA/CS 172: Python for Data Analysis

Nested Loops and Style

Prof. Travis Mandel

9/11/2024



UNIVERSITY
of HAWAII®

HILO

Announcements

- Lab 3 due today
- Lab 4 due Friday
- Friday – class will not meet
 - As stated in the syllabus, I will be off-island Friday (EPSCoR conference)
 - I will record a lecture/demo and post to Laulima (Loops wrap-up)
 - There will be no lab out on Friday

Agenda

- Nesting loops and if statements
- Some style notes

If statements within if statements

- Can put one if statement inside another
 - Called **nesting**
 - Make sure to indent!

if x > 0:

if y > 0:

print("x and y both positive")

#inside both

#outside one

#outside both



When nesting, careful with indentation!

```
if x > 0:
    if y > 0:
        print("x and y both
        positive")
    else:
        print("negative")
```

```
if x > 0:
    if y > 0:
        print("x and y both
        positive")
    else:
        print("negative")
```

What's the difference?

How will this affect how the code runs?

If statements within Loops

- Can nest an if inside a loop

```
for i in range(10):  
    if i == 3 or i == 7:  
        print("i is", i)
```



Loops within loops

- for i in range(5):
 - for j in range(5):
 - print("i", i ,"j", j)
 - #inside both
 - #inside i, outside j
- #outside both



Demo

- Nested loop to count all characters in a file

Indentation Error & Tab Error



- As we can see, indentation is key in Python
 - Different than most programming languages
- Python can give some very strange errors because of this
 - Indentation Error:
 - Python can't figure out why you indented or unindented
 - Tab Error:
 - You mixed tabs and spaces, Python can't figure out how to line things up
 - Usually only happens when switching editor/computers

Some indentation works, but doesn't look pretty

- Python doesn't care about the amount of the indent, just what it is relative to other blocks
- Small (one space) or large (10 tabs) is fine with python
 - Demo
- Good style is to keep all the indents a consistent size



Why do we care about style in data science?

- As data scientists, want to be able to share our work with others!
 - Want to make our code easy for other data scientists to read
 - Want to make it easy for others to come in and modify
 - Ideally, even want to make it easy for non-data-scientists to understand!
- So style is super important in DS!



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

Style tip 1: Make sure indentation looks nice

Style tip 2: Careful with things we didn't cover in class, or things we said not to do

- Bad style way:
 - `f = open(..)`
 - `f.write("This is a test string")`
 - `f.close()`
- Good style way:
 - `f = open(..)`
 - `print("this is a test string", file=f)`
 - `f.close()`
- You may not agree with me on everything – that's fine
- A big part of being a good programmer is learning the conventions and guidelines , and following them

Style tip 2: Minimizing re-looping, or closing/opening files

- Bad style way:
- `i = 0`
- `for k in range(20):`
 - `i += k`
- `j = 0`
- `for k in range(20):`
 - `j += k`

- Better way:
- `i = 0`
- `j = 0`
- `for k in range(20):`
 - `i += k`
 - `j += k`



Style tip 3: No “magic numbers” (or magic strings)

- Bad style way:

- `datapointsPerYear = 10*3*365`

- `surveysPerYear = 3*365`



- Better way:

- `daysPerYear = 365`

- `surveysPerDay = 3`

- `dataPointsPerSurvey = 10`

- `surveysPerYear = \`
`surveysPerDay * daysPerYear`

- `datapointsPerYear = \`
`dataPointsPerSurvey * surveysPerYear`

Style tip 4: No “dead code”

- Don't leave in lines that do little or nothing
- for line in f:
 - num = int(line) # dead code!
 - num = float(line)
- Variables that are never used, etc.



Lab out

- Practice with nested loops
- Keep style tips in mind for labs & projects